CSS4005 Final Project Report

> Copy this file into your repository as `REPORT.md`, fill every section, and export it to **PDF** for submission. Keep the headings: graders follow them with `RUBRIC.md`. Every claim needs **evidence**: a screenshot, command output, a log excerpt or a link to the exact file/line in your repo. Label screenshots with your `resource_prefix` visible.

---

0. Cover

Student	Full name, student ID
Course / stream	CSS4005-ENG-1 / ENG-2
Track	AWS (own account / Learner Lab) or GCP
Region / zones	e.g. `us-east-1`, zones `us-east-1a`, `us-east-1b`
Repository	`https://github.com/<you>/<repo>` (public)
Demo video (if torn down)	link
Submission date	

1. Variant applied

Fill in **your** values and show where each one is applied.

Field	My value	Where it is applied (file / screenshot)
`theme`		`APP_NAME`, item type
`resource_prefix`		resource names + tags
`vpc_cidr`		VPC + subnet plan
`traffic_pattern`		scaling policies, load test
`rpo`		backup / snapshot schedule, replication
`rto`		restore + failover drill timings
`monthly_budget_usd`		budget alert / lab spend
`db_engine`		primary + replica
`clickhouse_required`		ClickHouse node (or "no")
`cluster`		cluster build
`hpa_cpu_target`		HPA manifest
`password_min_length`		DB policy, app checker, CI test
`password_require_symbol`		same
`alert_cpu_pct`		CPU alarm
`alert_5xx_per_min`		5xx alarm
`backup_retention_days`		snapshot + logical backup retention

2. Architecture

2.1 Diagram

Insert your diagram: zones, subnets with CIDRs, LB, cluster nodes, DB primary/replica (+ ClickHouse), NAT, bucket, registry, secret store, monitoring, CI/CD.

2.2 Shared responsibility

Which parts are the cloud provider's job and which are yours **in your design** (note how self-hosting the DB moves work to you).

2.3 Architecture Decision Records (≥ 4)

For each: **Context → Decision → Alternatives considered → Consequences.**

3. Networking

- Subnet plan table (tier, zone, CIDR, route table / route).

- Security groups / firewall rules table: rule → source → port → why.
- Second layer (NACL / firewall policy) and why.
- Evidence: DB port unreachable from the internet and from the public tier.

4. Compute, Linux and the Kubernetes cluster

- Launch / instance templates and startup scripts (node + DB roles).
- Hardening: non-root services, patching, open ports (`ss -tulpn`), remote access without public SSH (SSM / IAP / bastion).
- Cluster: how it was built (your `cluster` variant), nodes per zone (`kubectl get nodes -L topology.kubernetes.io/zone`).
- Workload: Deployment (replicas, requests/limits, probes, non-root), zone spreading, Service + Ingress/LB path, ConfigMap, how the Secret is created from the secret manager.
- HPA with your `hpa_cpu_target` .
- Rolling update + rollback evidence (`kubectl rollout history`).
- Node / zone failure drill: what you did, pods rescheduled, user impact.

5. Load balancing and auto scaling

- LB listeners, health checks (`/readyz`), targets/backends.
- Pod-level and node-level scaling policies and why they fit your `traffic_pattern` .
- Load test: tool, scenario, graphs of requests, latency, pods, nodes.
- Self-healing demo (pod kill + node VM terminate).
- HTTPS: certificate approach.

6. Storage

- Bucket settings: versioning, lifecycle JSON, Block Public Access, encryption.
- Presigned / signed URL working and expiring.
- DB data disk (separate from OS), encryption, snapshot schedule + retention.

7. Self-hosted database

- Engine, versions, primary/replica placement (zones), replication setup and lag evidence.
- ClickHouse (if required): how the app uses it.
- Roles `admin` / `app_rw` / `app_ro` : grants + one **denied** action per limited role.
- Password policy: configuration + a weak password **rejected** (DB and `/api/admin/check-password`).
- Backups: snapshot + logical backup schedule; how it meets your RPO.
- **Restore test**: steps, data check, duration vs RTO.
- **Failover drill**: steps, downtime, data loss vs RPO/RTO.
- **Self-hosted vs managed (RDS / Cloud SQL)**: comparison table (backups, patching, failover, monitoring, cost, effort) and your conclusion.

8. Identity and secrets

- Roles / service accounts and what each can access (least privilege). Learner Lab: the policy JSON you *would* attach instead of `LabRole`.
- Which secrets live in the secret manager; how nodes/pods get them at boot/deploy; file permissions.
- Proof that no secret is in git, images, user data or Terraform outputs.
- Rotation plan (and rotation done, if you did the stretch).

9. Containers and CI/CD

- Image: Dockerfile highlights, registry, immutable tags, scan results.
- Pipeline stages (screenshot of a green run and a **failed** run that a check blocked).
- Deploy to the cluster + rollback evidence.
- How CI authenticates to the cloud and why.

10. Security checks — findings table

Check	Tool	Finding	Severity	Status (fixed / accepted + why)
Secret scan	gitleaks			
Image scan	Trivy / registry			
IaC scan	Trivy config / tfsec / Checkov			
Password policy	DB + app + CI test			
Network review	manual			
Storage review	manual			
IAM review	manual			
TLS	manual			

11. Infrastructure as Code

- Module structure (tree), state backend and how state is protected.
- `fmt` / `validate` in CI.
- **Reproducibility**: `destroy` → `apply` timing and result.

12. Monitoring and logging

- Dashboards (LB, app RED, nodes/pods, DB).
- Alerts (≥ 4) with thresholds, one shown **firing and resolving**.
- Centralized logs and the saved query following one `request_id`.

13. High availability, Well-Architected and cost

- Zone-failure drill results.
- Well-Architected review: one paragraph per pillar with evidence.
- Cost: Pricing Calculator estimate (24/7 month), actual spend, budget alert at `monthly_budget_usd`, ≥ 3 optimizations.
- Teardown proof.

14. Reflection

- The hardest problem and how you solved it.
- What you would do differently / next.
- Sources and tools used (including AI assistants and how you used them).